Backward Value Iteration on a 30×30 Grid

Two schedules of one Bellman equation, compared on cost, space and map difficulty, and run live in ROS 2 and RViz2

Kush Tokas / 2023296 / arp_a1_bvi / IIIT Delhi / 1 October 2026

54 / 54 / 56	18.5×	$\lvert X\rvert^{1.52} \rightarrow \lvert X\rvert^{1.03}$	PASS 5/5
J(start): easy / medium / hard optimal path length	fewer state updates, queue ABVI vs sync BVI (200 maps)	growth of the work, BVI → ABVI (grids 10^2 … 80^2)	optimality certificate, all 6 runs + independent ROS node · 434 tests

SECTION 1
The task and the model

A point robot moves on a **30×30 grid** with exactly **180 obstacles** (20 %), drawn uniformly at random from a seed; start (1, 1), goal (28, 28). Each move N, S, E or W to a free neighbour costs 1. The state set X is the free cells (720 per map), *Succ(s)* are the free neighbours of s, and *Pred(y)* = { p : y ∈ Succ(p) }. Moves are reversible, so Pred = Succ on this grid, but the two roles stay different: **Pred says who must be updated, Succ says how to compute the value.** The goal is the cost-to-go J of every state, not one path, so A*-style single-query search does not apply and none is used.

#	ASSIGNMENT REQUIREMENT	WHERE IT IS SHOWN
1	ROS 2 installed; 30×30 map with 180 random obstacles; robot placed at the start cell	`/map` OccupancyGrid, Humble in user space; robots start at START · §6, Fig. 6
2	Cost-to-go values J(s) visualised	heat-map + integer labels `/bvi_values` · Fig. 2, Fig. 6
3	Optimal path from start to goal	`/bvi_path`, `/abvi_path`; certificate C5 · §7
4	Robot navigates along the path in RViz2	two animated robots, 1 cell / 0.4 s, J falls by 1 per step · Fig. 6
+	Two BVI variants compared; easy / medium / hard maps	§2 algorithms · §3 maps · §5 cost and space

SECTION 2
One equation, two schedules

$$J(g) = 0, \quad J(s) = \min_{s' \in \mathrm{Succ}(s)} [\, 1 + J(s') \,], \quad J = \infty \text{ on obstacles and unreachable cells} \tag{1}$$

$$\pi(s) = \operatorname{argmin}_{s' \in \mathrm{Succ}(s)} [\, 1 + J(s') \,], \quad \text{ties broken N, S, E, W; path = follow } \pi \text{ from START to g} \tag{2}$$

ALGORITHM 1 · SYNCHRONOUS BVI (JACOBI SWEEPS)

```
1. J₀(s) ← ∞  ∀ s ∈ X;   J₀(g) ← 0
2. repeat   # sweep i = 1, 2, …
3.     for each s ∈ X \ {g}
4.         Jᵢ(s) ← min_{s′∈Succ(s)} [ 1 + Jᵢ₋₁(s′) ]
5. until Jᵢ = Jᵢ₋₁              # a full sweep changed nothing
6. return J, π(s) = argmin_{s′} [ 1 + J(s′) ]
```

ALGORITHM 2 · QUEUE ABVI (FIFO WAVEFRONT)

```
1. J(s) ← ∞  ∀ s;   J(g) ← 0;   Q ← ⟨ g ⟩
2. while Q ≠ ∅
3.     y ← Q.popleft()
4.     for each p ∈ Pred(y), p ≠ g
5.         v ← min_{s′∈Succ(p)} [ 1 + J(s′) ]     # live values
6.         if v < J(p):   J(p) ← v;   Q.append(p)
7. return J, π
```

A backs up every state in every sweep using only the previous sweep's values, so it needs max $J + 1$ sweeps and keeps two value arrays. **B** backs up only the predecessors of a state whose value just changed and uses new values immediately. A FIFO queue is sufficient because every cost is 1: states are finalised in non-decreasing J, each is written exactly once, and the queue holds at most two adjacent J levels (all asserted in the tests). Both are hand-written in `bvi_core/bvi.py`; no A*, Dijkstra, NetworkX or Nav2 is called.

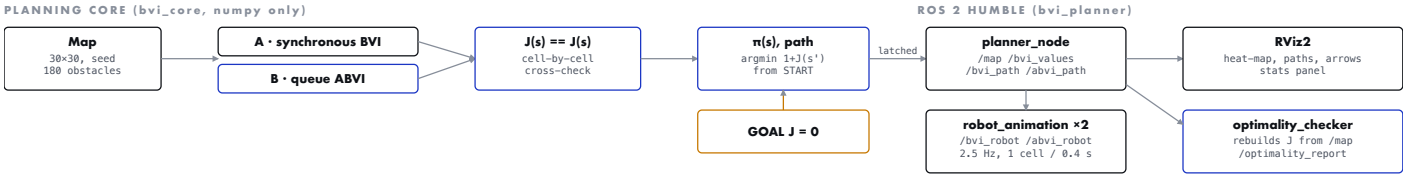


FIGURE 1 · PIPELINE. The same pure-Python core feeds the standalone tools and the ROS 2 package (one copy, linked into the package). The two J maps are compared cell by cell before anything is published; ● amber marks the goal.

SECTION 3

Easy, medium and hard maps

Difficulty is chosen by a fixed rule, not by hand. Seeds 0–499 give one 180-obstacle map each; **480** have START connected to GOAL and **20** are rejected and counted. Maps are ranked by (*stretch* = J(start) / Manhattan, *turns*, seed) and the 20th, 50th and 90th percentiles are named easy, medium, hard. Stretch measures how far obstacles force the path above the Manhattan lower bound; turns measure how often it bends.

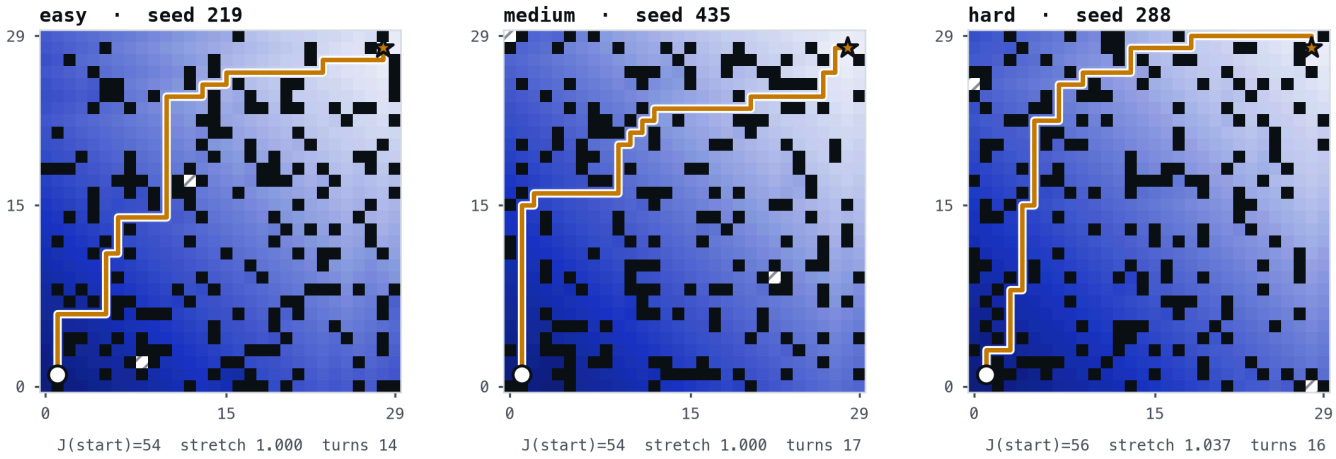


FIGURE 2 · THE THREE MAPS. Heat-map is J (dark = far from the goal), black cells are obstacles, hatched cells are free but unreachable, the amber line is the optimal path, ○ is START and the amber star is GOAL. The queue-ABVI path is shown; the BVI path has the same cost.

MAP	SEED	J(START)	MANHATTAN	STRETCH	TURNS	STATES WITH FINITE J	FREE BUT UNREACHABLE
easy	219	54	54	1.000	14	718	2
medium	435	54	54	1.000	17	718	2
hard	288	56	54	1.037	16	718	2

HONEST READING OF "DIFFICULTY"

With START and GOAL in opposite corners and 20 % uniform obstacles a monotone staircase path almost always exists: stretch has min 1.000, median 1.000, max 1.111. Easy and medium therefore differ only through the turn count (14 vs 17); hard is the first level with a detour (+2 moves, 1.037). These levels describe **navigation** difficulty. They barely change BVI's **computation**: the full J is solved regardless, so updates are 40,983, 40,983 and 42,421 for the synchronous variant (3.5 % spread).

SECTION 4

How the value wave moves

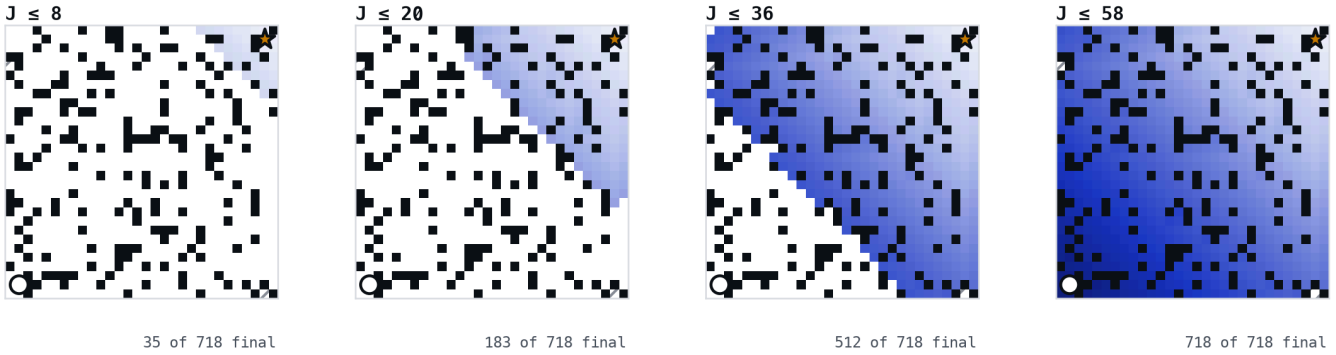


FIGURE 3 · QUEUE ABVI ON THE HARD MAP. Cells whose J is already final after the wave has passed level d. Because FIFO with unit costs finalises states in non-decreasing J, the finite region is exactly { J ≤ d }: a diagonal front sweeping out from the goal that is squeezed through the obstacle gaps. Hatched corner cells are never reached (J stays ∞).

This picture is the whole difference between the algorithms. ABVI touches the front: the 29 states in the queue at its widest (peak queue) out of 718. Synchronous BVI keeps sweeping all 719 states, including the ∞ region ahead of the front and the finished region behind it, for 59 sweeps (= max J + 1). Both stop at the same fixed point of (1), which is unique, so the J maps are identical on every cell of every map we ran.

WHAT WAS CHECKED, AND WHAT IT PROVES

- **Equality:** sync J = queue J = edge-relax J on 60 test maps, 200 benchmark maps and all demos (cell by cell, ∞ = ∞).
- **Path:** every step is a legal move, J falls by exactly 1 per step, and the path has J(start) moves.
- **Failure injection:** the tests corrupt one J value, take a detour, or give an unreachable cell a finite J; the certificate (§7) rejects each.

SECTION 5

Six runs: {easy, medium, hard} × {BVI, ABVI}

MAP	ALGORITHM	ITERATIONS	STATE UPDATES	SUCCESSOR EVALS	RUNTIME MS (MAC)	J WORDS STORED	PEAK HEAP KIB	J(START)	TRAVEL S	CERT.
easy	BVI sync	57 sweeps	40,983	126,768	29.75	1,800	132	54	21.6	PASS
easy	ABVI queue	718 pops	2,224	7,372	**2.01**	934	143	54	21.6	PASS
medium	BVI sync	57 sweeps	40,983	127,452	29.76	1,800	132	54	21.6	PASS
medium	ABVI queue	718 pops	2,236	7,420	**2.01**	928	143	54	21.6	PASS
hard	BVI sync	59 sweeps	42,421	130,921	30.81	1,800	132	56	22.4	PASS
hard	ABVI queue	718 pops	2,219	7,323	**1.99**	929	143	56	22.4	PASS

Updates = full backups attempted; successor evals = neighbour look-ups inside them; runtime = best of 5, CPython 3, single thread; travel = path cost × 0.4 s per move. **J words stored** is the algorithmic working set: BVI holds J_{i-1} and J_i (2 × 900), ABVI holds one J array plus its queue peak. **Peak heap** is the measured Python allocation (tracemalloc) of one whole run including the policy dictionary and the convergence trace, which ABVI records per pop (718 entries vs 57); this bookkeeping hides the algorithmic saving, so both are shown.

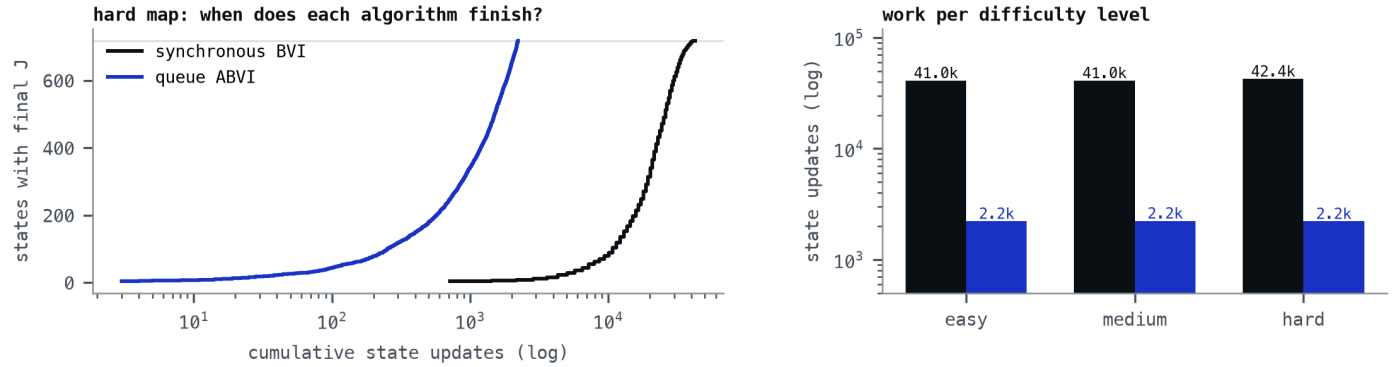


FIGURE 4 · COST. Left: states with final J against cumulative updates (log axis) on the hard map; ABVI finishes after 2,219 updates, BVI after 42,421. Right: updates per map. Cost and travel time of the resulting path are identical for both algorithms.

COST

On 200 random reachable maps ABVI needs **18.5×** fewer updates and **17.2×** fewer successor evaluations; the Bellman-backup variant runs in 2.03 ms against 30.19 ms (15×) on the Mac, and 3.3 ms against 51.0 ms on the Linux ROS machine. Updating by edge relaxation, J(p) ← min(J(p), 1 + J(y)), gives the same J with 2222 look-ups (0.84 ms). Update counts are hardware-independent; the millisecond figures are only CPython timings.

Why: BVI does |X| backups × (max J + 1) sweeps, and max J grows like the grid side $\sqrt{|X|}$, so its work scales as $|X|^{1.5}$. ABVI pops each state once and touches ≤ 4 predecessors per pop: about 4|X| work.

SPACE

BVI needs two value arrays (2|X| words) because a sweep reads J_{i-1} while writing J_i; ABVI needs one array plus the queue, which holds only the wavefront: peak **34** / 28 / 29 entries on the three maps, ≈ 5 % of the states. The queue peak grows only as $|X|^{0.51}$ (the front is a curve across the grid), so ABVI's extra memory is $O(\sqrt{|X|})$ while BVI's extra array is $O(|X|)$. Both return the same policy dictionary (≈ 717 entries), which dominates the real heap.

SECTION 6

Scaling beyond 30×30

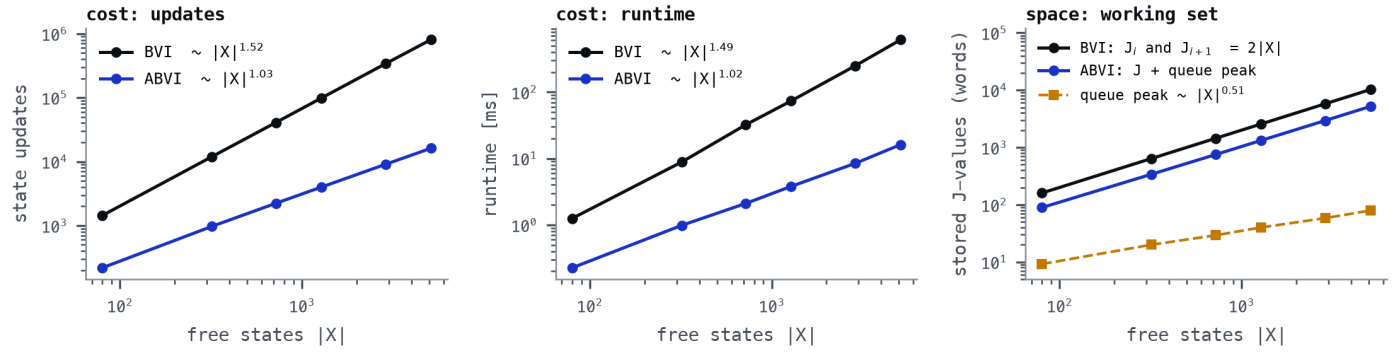


FIGURE 5 · SCALING. Square grids of side 10, 20, 30, 40, 60, 80 with 20 % obstacles, corner to corner, mean over 5 connected maps each. Least-squares slopes on the log-log axes: updates $|X|^{1.52}$ (BVI) vs $|X|^{1.03}$ (ABVI); runtime $|X|^{1.49}$ vs $|X|^{1.02}$.

The gap widens with size: at 5,120 free states (80×80) BVI needs 804,707 updates against 16,148 (50×) and 614 ms against 16 ms. The measured exponents match the analysis (1.5 and 1.0), so the 18× at 30×30 is not a constant factor: it is the ratio of the sweep count to a constant.

SECTION 7

ROS 2 and RViz2: what runs

The package `bvi_planner` (rclpy, ament_python) has three nodes. `planner_node` builds the map, solves both BVIs and publishes everything once with latched (transient-local) QoS; `robot_animation_node` (two instances) walks each path one cell per 0.4 s and restarts after a 3 s hold at the goal; `optimality_checker_node` subscribes to `/map` and the paths, rebuilds J independently and publishes the verdict. ROS 2 Humble runs on Linux (RoboStack/conda or apt); RViz2 renders the live scene.

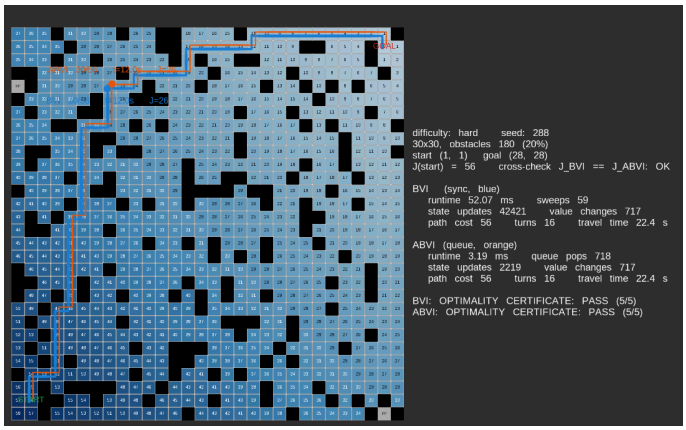


FIGURE 6A · RVIZ2, ROBOTS MID-RUN. Obstacles, J heat-map with labels, blue BVI and orange ABVI paths, two robots (label: step/56, t, J), stats panel with both certificates (hard map, real screenshot).

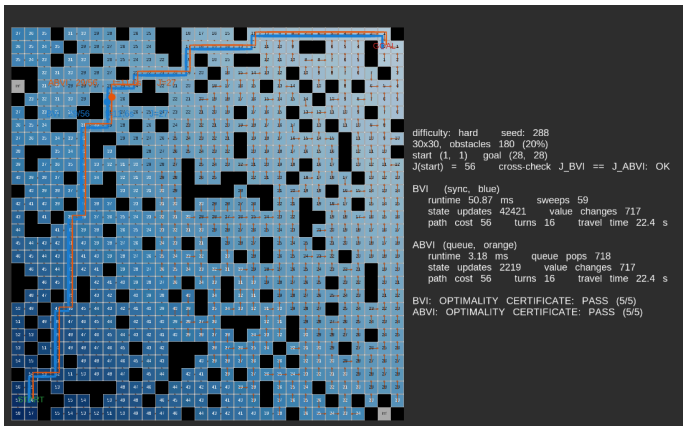


FIGURE 6B · POLICY ARROWS. The optimal action from *every* reachable cell (ABVI policy), not only along one path.

TOPIC	TYPE	CONTENT
`/map`	nav_msgs/OccupancyGrid	0 free, 100 obstacle, 30×30, 1 m cells, frame `map`
`/bvi_values`	MarkerArray	J heat-map (CUBE_LIST), integer TEXT labels, START / GOAL
`/bvi_path /abvi_path`	nav_msgs/Path	optimal path from sync BVI / queue ABVI
`/bvi_policy /abvi_policy`	MarkerArray	$\pi(s)$ arrow for every reachable state
`/bvi_robot /abvi_robot`	MarkerArray	animated robots, 2.5 Hz, label step / total, time, remaining J
`/bvi_stats`	Marker	comparison panel and both certificates
`/optimality_report`	std_msgs/String	verdict of the independent checker node

SECTION 8

Verification

OPTIMALITY CERTIFICATE · BVI_CORE/VERIFY.PY · ALL SIX RUNS + ROS CHECKER

C1 boundary	**J(g)=0, obstacles ∞**
C2 Bellman fixed point	**max |J − min(1+J′)| = 0 over 719 states**
C3 finite ⇔ reachable	**718 reachable, 2 unreachable free (hard)**
C4 lower bound	**|J(u) − J(v)| ≤ 1 on every edge; J ≥ Manhattan**
C5 path optimal	**legal, moves = J(start) = 56 (hard)**

CHECK	RESULT
unit + integration tests (macOS and Linux)	434 passed
sync J = queue J = relax J	identical, all maps
ROS launch easy / medium / hard	both robots reach the goal
C1–C5, planner and checker node	PASS 5/5 on both paths
colcon build, RViz2 launch	clean build, no errors

C4 and C5 together prove optimality: no path from s can have fewer than J(s) moves, and the extracted path has exactly J(start). C1–C3 prove J is the unique optimal cost-to-go.

ASSUMPTIONS AND LIMITS

- Cells are (x, y) with (0, 0) bottom-left, as in OccupancyGrid; resolution 1 m. Equal-cost ties break N, S, E, W, so other optimal paths exist.
- The robot is a marker moving cell by cell (kinematic animation); there is no dynamics simulation.
- Difficulty levels rank navigation, not BVI cost (§3). Timings are CPython; the counts are the portable result.
- Disconnected maps are rejected by a documented rule (seed, seed+1, …) and logged, never hidden.

RUN IT

`./scripts/build.sh && ./scripts/verify.sh` · `./scripts/vnc_start.sh` · on the Mac `./scripts/mac_view.sh` · `./scripts/demo.sh hard`. Full step-by-step manual with the TA script: `DEMO_SERVER.md`.

REFERENCES

[1] R. Bellman, *Dynamic Programming*, Princeton University Press, 1957.
[2] S. M. LaValle, *Planning Algorithms*, Cambridge University Press, 2006, ch. 2 (discrete planning, value iteration).
[3] CSE634 lecture notes: backward value iteration and asynchronous BVI (sessions 4–6), IIIT Delhi, August 2026.
[4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics* 7(66), 2022.